Orion — Agent Planner Subsystem

A plain-language guide with Input → Processing → Output and live, code-generated proof. Codebase: `mabgp` | Layer: `backend/app/planning/` | 1083 tests passing, fully offline

1. What the Planner Subsystem is

Orion turns one sentence into a full business plan using 21 specialised AI agents across 5 departments. The **Planner Subsystem** is the layer that, **before any agent runs**, turns a confirmed request into an **explicit execution plan**: a graph of tasks that shows exactly which agents will run, in what order, which depend on which, and what each will cost. It then **executes** that plan, tracks each task's status, saves progress after the run, and records an audit trail.

> **In one line:** the report says "here is what *would* run"; the planner produces the graph, runs it, and records "here is what *is* running / has run" — reusing Orion's existing execution engine, with no new AI model.

2. Input → Processing → Output

```
   INPUT                    PROCESSING                                    OUTPUT
  -----                    ----------                                   ------
confirmed report  --->  compile_plan()   (deterministic, no LLM)  --->  ExecutionPlan
(prompt + intent        - project 21/17 agents from the topology        - nodes with status
 + agent roster)        - wire depends_on edges (the task DAG)          - dependency edges
                        - attach tier / tokens / grounding              - est. token budget
                             |
                        run_plan()       (reuses orchestrate)     --->  live node statuses
                        - policy budget check (fail-closed)             - completed/failed
                        - execute the DAG, map results to nodes         - checkpoint saved
                        - checkpoint + append events                    - plan_events trace
```

Input (what the operator provides)

- **A confirmed planning report** — created earlier from a one-line prompt (e.g. an ice-cream shop), reviewed, and explicitly approved by the operator.
- The report already carries the classified **intent**, the **routing decision** (which departments apply), and the **agent roster** (the 21 or 17 agents).

Processing (what the code does)

- **compile_plan()** — deterministically turns the flat agent roster into a graph. It fills each task's `depends_on` edges from the department stage layout. **No LLM call.**
- **ExecutionPolicy** — one frozen limits object (max tokens, timeout, repair rounds). It **fails closed**: a missing limit stops the run rather than guessing.
- **run_plan()** — reuses Orion's existing `orchestrate()` engine to run the DAG, maps each agent's result back to its node (completed / failed / skipped), enforces the budget, and writes a checkpoint + an append-only event trace to SQLite.

Output (what the operator gets)

- An **ExecutionPlan** object with every task's status, dependencies, and token usage.
- A **saved checkpoint** (crash-safe: an interrupted run resumes from the last saved point).

- An **append-only event trace** (`plan_events`) — a reproducible record for auditing.

3. Live proof — real output for different user types

The tables below are produced by running the actual `compile_plan()` code on two different prompts. This is not a mock-up — it is the real graph the subsystem generates.

Physical business (retail / F&B / clinic)

Operator prompt (input)	`"Ice cream shop in Andheri, Rs 15L budget"`
Plan nodes produced	**21** agents
Initial plan status	draft (all nodes pending)
Estimated max tokens (upper bound)	23,200
Stage-1 entry nodes (no dependencies)	market_research, competitive_analysis, location_intelligence, revenue_model, cost_structure, brand_naming, buyer_persona, supply_chain, staffing_plan, tech_stack

Agent (node id)	Dept	Stage	Run	Tier	Max tok	Grounding	Depends on
`market_resea`	strategy	1	parallel	reasoning	900	serper	—
`competitive_`	strategy	1	parallel	reasoning	1000	openstreetmap	—
`location_int`	strategy	1	parallel	fast	900	LLM-only	—
`business_pla`	strategy	2	parallel	reasoning	1200	LLM-only	`market_research, competitive_analysi location_intelligen`
`risk_analysi`	strategy	2	parallel	reasoning	1300	LLM-only	`market_research, competitive_analysi location_intelligen`
`revenue_mode`	finance	1	parallel	reasoning	1200	LLM-only	—
`cost_structu`	finance	1	parallel	reasoning	1400	LLM-only	—
`funding_stra`	finance	2	parallel	fast	900	LLM-only	`revenue_model, cost_structure`
`pl_forecast`	finance	2	parallel	reasoning	800	LLM-only	`revenue_model, cost_structure`
`brand_naming`	branding	1	sequential	fast	1000	LLM-only	—
`brand_identi`	branding	2	parallel	fast	900	LLM-only	`brand_naming`
`brand_voice`	branding	2	parallel	fast	1000	LLM-only	`brand_naming`
`visual_desig`	branding	2	parallel	fast	700	LLM-only	`brand_naming`
`buyer_persor`	marketing	1	sequential	reasoning	1500	LLM-only	—
`digital_plar`	marketing	2	parallel	reasoning	1300	LLM-only	`buyer_persona`

campaign_pla	marketing	2	parallel	fast	1300	LLM-only	buyer_persona
seo_content	marketing	2	parallel	fast	1000	LLM-only	buyer_persona
supply_chain	operations	1	parallel	reasoning	1200	LLM-only	business_planning, risk_analysis, funding_strategy, pl_forecast, brand_identity, brand_voice, visual_design, digital_plan, campaign_plan, seo_content
staffing_pla	operations	1	parallel	reasoning	1300	LLM-only	business_planning, risk_analysis, funding_strategy, pl_forecast, brand_identity, brand_voice, visual_design, digital_plan, campaign_plan, seo_content
tech_stack	operations	1	parallel	fast	1100	LLM-only	business_planning, risk_analysis, funding_strategy, pl_forecast, brand_identity, brand_voice, visual_design, digital_plan, campaign_plan, seo_content
process_flow	operations	2	sequential	fast	1300	LLM-only	supply_chain, staffing_plan, tech_stack

Digital business (online / SaaS / app)

Operator prompt (input)	`"Online tutoring platform for CBSE students"`
Plan nodes produced	**17** agents
Initial plan status	draft (all nodes pending)
Estimated max tokens (upper bound)	18,300
Stage-1 entry nodes (no dependencies)	market_research, competitive_analysis, location_intelligence, revenue_model, cost_structure, brand_naming, buyer_persona

Agent (node id)	Dept	Stage	Run	Tier	Max tok	Grounding	Depends on
market_resea	strategy	1	parallel	reasoning	900	serper	—

competitive_	strategy	1	parallel	reasoning	1000	openstreetmap	—
location_int	strategy	1	parallel	fast	900	LLM-only	—
business_pla	strategy	2	parallel	reasoning	1200	LLM-only	market_research, competitive_analysi location_intelligen
risk_analysi	strategy	2	parallel	reasoning	1300	LLM-only	market_research, competitive_analysi location_intelligen
revenue_mode	finance	1	parallel	reasoning	1200	LLM-only	—
cost_structu	finance	1	parallel	reasoning	1400	LLM-only	—
funding_stra	finance	2	parallel	fast	900	LLM-only	revenue_model, cost_structure
pl_forecast	finance	2	parallel	reasoning	800	LLM-only	revenue_model, cost_structure
brand_naming	branding	1	sequential	fast	1000	LLM-only	—
brand_identi	branding	2	parallel	fast	900	LLM-only	brand_naming
brand_voice	branding	2	parallel	fast	1000	LLM-only	brand_naming
visual_desig	branding	2	parallel	fast	700	LLM-only	brand_naming
buyer_persor	marketing	1	sequential	reasoning	1500	LLM-only	—
digital_plar	marketing	2	parallel	reasoning	1300	LLM-only	buyer_persona
campaign_pla	marketing	2	parallel	fast	1300	LLM-only	buyer_persona
seo_content	marketing	2	parallel	fast	1000	LLM-only	buyer_persona

> **Why the two differ:** a **physical** business needs Operations (supply chain, staffing, tech, process) so it gets **21 nodes**; a **digital** business has no physical operations, so Operations is skipped and it gets **17 nodes**. The operator sees exactly which 4 agents were dropped — the routing is transparent, not hidden.

4. How dependencies work (the key new capability)

- **Stage 1** agents (e.g. market research) start immediately — empty `depends_on`.
- **Stage 2** agents wait for their own department's Stage 1 (e.g. business planning waits for the three strategy agents).
- **Operations** is a cross-department join: it waits for **all** of Stage 2 across every other department before it begins — the same barrier Orion's execution graph enforces.

5. Safety & guarantees

Deterministic	Same confirmed report → identical plan every time (verified by a golden test).
Budget (fail-closed)	max tokens 1,000,000; warn at 800,000; a run over budget is refused before it starts.

Crash recovery	Plan is checkpointed to SQLite; an interrupted run resumes from the last saved node — no external infrastructure.
Honest status	The plan mirrors the run's true verdict (completed / partial / failed); nothing is faked.
Offline & free	No new dependencies, no paid API tier; the whole test suite runs with zero network access.

Generated by `backend/_planner_pdf.py` from live `app.planning.compile` output. Orion (mabgp), University of Mumbai final-year project.